

Container name or container ID.  
If the container name is returned - container started.

⇒ `docker ps -a`

(here status would be the seconds  
it was started ago).

1) To list only the running containers

⇒ `docker ps`

### Day 3

⇒ Docker Desktop is an LT.

⇒ If laptop is shutdown when container is running  
after restarting the laptop the docker containers  
will not show when listing it.

⇒ To start it

\* `docker start demon`

⇒ Now list it

> `docker ps`

⇒ To stop it

> `docker stop demon`

⇒ Now list it

> docker ps

⇒ To remove it

> docker rm demoncon.

⇒ Now list it > docker ps -a.

⇒ Starting it with container ID.

> docker start cont ID.

⇒ while running trying to remove it.

> docker rm cont ID (Not possible).

⇒ But we can force <sup>remove</sup> ~~stop~~ it.

> docker rm -f cont ID.

⇒ Now list & see if

> docker ps -a.

⇒ To create new container without create Cmd.   
 <sup>creates only & idle.</sup>   
 daemon. (Background process).

> docker run -d --name sji-devops httpd

(It creates and launches)

↓   
 custom name

flag for

↓   
 image name

⇒ without -d. it will create but container will not show - occupies client UI   
 but on entering ctrl+c the container will also shutdown.

⇒ Again start & list it - it will show that the container is running which is created without d.

→ To open the created container.

> docker exec -it sji-devops /bin/bash

↓  
interactive terminal.

opens like

root @ cont ID -

⇒ To exit

> exit.

→ Again open it with root user.

→ then update.

> apt update

→ Going to apache root directory to configure the source code.

> ls

> cd htdocs.

> ls

> cat index.html.

> exit.

> ipconfig (Take WLAN-IPV4 address)

→ Assigning the host code to the container.

> docker run -d --name test -p 8082:80 httpd.

→ On browser paste the copied address with host code.

eg: http://ipadd:8082.

→ To edit the source code.

→ ~~docker~~ docker ~~run~~ exec -it test /bin/bash.

# cd htdocs

# ls

# apt ~~install~~ update.

# apt install vim -y.

↳ default yes.

# vi index.html.

→ In editor edit the body.

:wq →

# cat index.html.

Task launch container named task - apache image.  
port: 82.

take  
→ To backup of container as image

> docker commit task.  $\frac{\text{snapping: 1.0}}{\downarrow \text{img name} \quad \downarrow \text{tag (X)}}$

→ To check whether image present or not

> docker images.

→ Launching a new container with snapping taken by

> docker run -d --name demo0 -p 8083:80

(In another port  $\frac{\text{snapping: 1.0}}{82 \rightarrow 8083}$ )

→ To store the backup image in laptop as a file to share it because now it is in docker engine (local repo)

> docker save snapping:1.0 → demo0backup.tar  
> ls. ↳ filename

→ difficult to load in docker so,

> docker save -o sjibackup.tar snapping:1.0

> ls.

→ To remove images.

> docker rmi snapimg:1.0.

→ force remove images.

} but the container will not be deleted.

~~docker rmi -f~~

> docker ~~rmi~~ rmi -f snapimg:1.0

→ To load the backup image.

> docker load -i sjibackup.tar

> docker images

~~> docker exec it task /bin/bash~~

Building docker image: Step 1 automation.

→ On dockerfile → Instructions should be capital letters arguments can be small letters.

Syntax: INSTRUCTIONS arguments.

→ Dockerfile:

Docker can build images automatically by reading the instructions from a Dockerfile. A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

1) FROM:

create a new build stage from a base image.

2) RUN

execute build commands

3) COPY

COPY index.html /var/www/html/  
          ↓                          ↓  
          source                  destination.

⇒ EXPOSE

→ CMD:

Specify default commands / can be overridden

→ LABEL

→ ENV.

→ ENTRYPOINT:

Specify default executable (can't be overridden).

> explorer

↳ current working directory

Create docker.txt

• type script

> cat dockerfile.txt

> notepad index.html

type random → Save

> docker build -t customimg:latest

↓  
tag

> mv dockerfile.txt dockerfile

> docker build -t customimg:latest

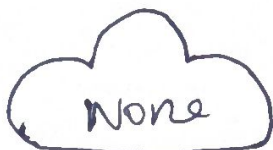
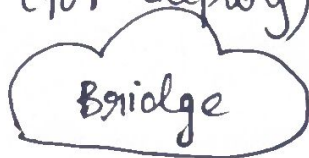
> docker images

> docker run -d --name filetest -p 8081:80  
customimg

Open in browser & check.

# Docker Networks

(for deploy)



(for test)

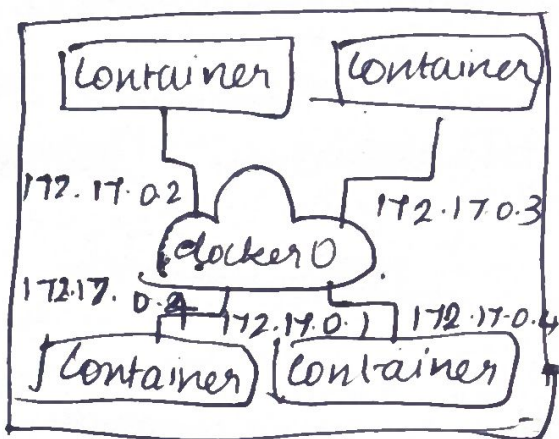


```
docker run ubuntu
```

```
docker run \
--network = none
Ubuntu
```

```
docker run \
--network =
host ubuntu
```

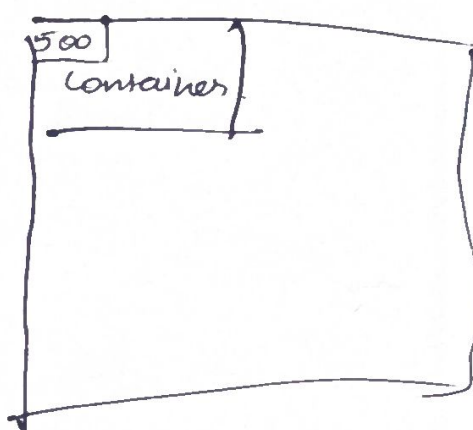
Docker Host



Docker Host



Docker Host



of containers.

→ Default launch on bridge network.